Data Structures

Stacks, Queues & Deques

Stacks & Queues

- Both can be implemented as dynamic array or linked list
- Stack
  - Last-in First-out (LIFO)
  - Push / pop / peek
- Queue
  - First-in First-out (FIFO)
  - Enqueue / Deque respectively
  - Similar interface to stack
  - Implementation varies significantly

Stacks

- Stack
  - Last-in First-out (FIFO)
  - Push/pop
  - Can be implemented as dynamic array or linked list
- Array-based implementation
  - Provide quick access, O(1), to the end
  - Costly re-sizes O(n)
- Linked list-based implementation
  - Access to the front of the list via head pointer
  - No re-size penalty

Stack Interface

```
Struct stack {
        int size;
        int *st;
or
        link *head;
};
Void initStack(struct stack *s, int cap);
Void pushStack(struct stack *s, TYPE val);
TYPE popStack(struct stack *s);
TYPE peekStack(struct stack *s);
Int isEmptyStack(struct stack *s);
```

Array Implementation of a Stack

- Push 1, 2, 3
- Pop
- Push 4, 5
- Pop
- Push 6
- Draw it out



Linked List Implementation of a Stack

- Push 1, 2, 3
- Pop
- Push 4, 5
- Pop
- Push 6
- Draw it out

Queues

- Queue
  - First-in First-out (FIFO)
  - Enqueue / Deque
  - Can be implemented as dynamic array or linked list
- Array-based implementation
  - Provides quick access, O(1), to both ends
  - Costly re-sizes O(n)
- Linked list-based implementation
  - Need access to both ends
  - No re-size penalty

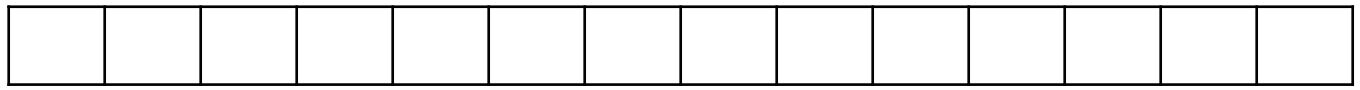
Queue Interface

```
Struct queue {
        int size;
        int *q;
or
        Link *head;
        Link *tail;
};
Void initStack(struct queue *q, int cap);
Void enqueue(struct queue *q, TYPE val);
Void dequeue(struct queue *q);
TYPE peek(struct queue *q);
Int isEmptyStack(struct queue *q);
```

Array Implementation of a Queue

- Enqueue 1, 2, 3
- Dequeue
- Dequeue
- Enqueue 4,5
- Dequeue

- Draw out pictures



Linked List Implementation of a Queue

- Enqueue 1, 2, 3
- Dequeue
- Dequeue
- Enqueue 4,5
- Dequeue

- Draw out pictures

Deque

- Deque
  - Access to front and back
  - AddFront / AddBack / GetFront / GetBack
  - Array-based implementation
    - Provide quick access - O(1) - to both ends
    - Costly re-sizes O(n)
  - Linked list-based implementation
    - Need access to both ends
    - No re-size penalty

Deque Interface

```
Struct deque {
        int size;
        int *q;
or
        Link *head;
        Link *tail
};
Void initDeque(struct deque *d, int cap);
Void pushFront(struct deque *d, TYPE val);
Void pushBack(struct deque *d, TYPE val);
void popFront(struct deque *d);
void popBack(struct deque *d);
TYPE getFront(struct deque *d);
TYPE getBack(struct deque *d);
Int isEmptyStack(struct deque *d);
```

Improved Array Implementation of a Queue

- Is there a way to improve on what we have talked about?
  - We can get rid of shifts
  - Track offset and size and capacity
- Enqueue 1, 2, 3
- Dequeue
- 4, 5
- Dequeue
- Draw out pictures



Linked List vs Dynamic Array

- What is the best choice?
  - Bag
  - Stack
  - Queue

Recursion

- Calling a function from within itself
- Often times the simplest way to express a complex task
  - Each time the function is called, a new copy of the variables is created
  - Not memory efficient but often times more readable
- Base case needs to be reached to end recursive calls
- We will do a lot of this eventually

Recursion Example

- Base case is x == 0
  - Ends recursion

```
Int factorial(int x) {
        if(x == 0) {
                return 1;
        }
        return(x* factorial(x-1));
}

Printf(“%d”, factorial(5));
```